
title: “Semana 1 — Sistemas de Numeración y Códigos de Error”
subtitle: “Arquitectura de Computadoras · FING UdelaR ·
Cronograma 03/08 - 07/08/2026”

Semana 1 — Sistemas de Numeración y Códigos de Error

Cronograma oficial de esta semana: Notas sobre Sistemas de Numeración · Notas sobre Códigos y Errores · Práctico 0 · Taller 0

Primer Parcial: jueves 24/09/2026 — quedan ~7 semanas de cronograma antes del parcial.

1. Qué hay que saber al terminar esta semana

- ☐ Escribir el polinomio característico de un número en cualquier base.
 - ☐ Convertir de cualquier base a base 10 (Caso A — aritmética de la base destino).
 - ☐ Convertir de base 10 a cualquier base (Caso B — aritmética de la base origen, divisiones/multiplicaciones sucesivas).
 - ☐ Convertir con parte fraccionaria en ambos sentidos, sabiendo truncar donde lo pida el enunciado.
 - ☐ Convertir binario ↔ octal / hexadecimal agrupando en bloques de 3 / 4 bits.
 - ☐ Resolver ecuaciones o determinar la validez de operaciones en una base desconocida.
 - ☐ Calcular la distancia de Hamming entre dos códigos y sus propiedades.
 - ☐ Calcular la cantidad mínima de bits de redundancia para un sistema de k bits.
 - ☐ Calcular y verificar bits de paridad (par/impar), incluyendo paridad horizontal/vertical (bidimensional).
 - ☐ Explicar y aplicar el código Entrelazado 2 de 5 (por qué detecta pero no corrige).
 - ☐ Codificar, detectar y corregir un error de 1 bit con código de Hamming (construir la tabla de posiciones, calcular los bits de paridad, calcular los síndromes al recuperar).
 - ☐ Calcular un checksum simple por suma binaria despreciando acarreo.
-

2. Teoría — Sistemas de Numeración

2.1 REPRESENTACIÓN POSICIONAL

Todo número N en base b se representa por su **polinomio característico**:

$$N = a_n \cdot b^n + a_{n-1} \cdot b^{n-1} + \dots + a_1 \cdot b^1 + a_0 \cdot b^0$$

Donde a_i es un dígito del sistema y b es la base (cantidad de símbolos del sistema).

2.2 CONVERSIÓN DE BASES — CASO A (ARITMÉTICA DE LA BASE DESTINO B)

Útil para **cualquier base** → **base 10**. Se expresan los símbolos y la base origen B en aritmética de b, y se evalúa el polinomio directamente.

Ejemplo: $A2F_{16} = 10 \cdot 16^2 + 2 \cdot 16^1 + 15 \cdot 16^0 = 2607_{10}$

2.3 CONVERSIÓN DE BASES — CASO B (ARITMÉTICA DE LA BASE ORIGEN B)

Útil para **base 10** → **cualquier base**. Se divide sucesivamente N entre b (con aritmética en B); los restos, del último al primero, son los dígitos.

N		b	
a0		N1	b
		a1	N2 b
			a2 ...

2.4 PARTE FRACCIONARIA

- **Caso A:** igual que enteros pero con potencias negativas ($a_{-1} \cdot b^{-1} + a_{-2} \cdot b^{-2} + \dots$).
- **Caso B:** se multiplica sucesivamente la parte fraccionaria por b (aritmética en B); la parte entera de cada resultado es el dígito. Se hacen tantos pasos como el enunciado pida — **no hay que llegar a 0 necesariamente**, muchas fracciones no terminan.

2.5 ATAJO BINARIO ↔ OCTAL / HEXADECIMAL

Como $8 = 2^3$ y $16 = 2^4$: agrupar el binario en bloques de 3 (octal) o 4 (hex) bits y traducir cada bloque por separado. Es mucho más rápido que pasar por base 10.

2.6 BASES DESCONOCIDAS

Cuando el enunciado da una ecuación u operación en una base x a determinar: plantear el polinomio característico con x como incógnita y resolver, o probar qué bases hacen válida una operación aritmética dada. Recordar: los dígitos usados en la operación deben ser símbolos válidos de esa base (ej. un dígito “8” no puede existir en base 8), y el resultado debe verificar la aritmética posicional.

3. Teoría — Códigos y Errores

3.1 DISTANCIA

$d(a,b)$ = cantidad de bits distintos entre dos códigos. Propiedades: $d(a,b) = d(b,a)$; $d(a,b) = 0$ si y solo si $a = b$; desigualdad triangular $d(a,b) + d(b,c) \geq d(a,c)$. La **distancia de un código** es la mínima distancia entre dos valores válidos cualesquiera del código.

- Capacidad de **detectar** e errores: $d \geq e + 1$
- Capacidad de **corregir** e errores: $d \geq 2e + 1$

3.2 BITS DE REDUNDANCIA

Para un sistema de k bits de datos con p bits de redundancia, corrección de 1 bit exige:

$$2^p \geq p + k + 1$$

(el +1 es para poder indicar también “sin error”).

3.3 PARIDAD

Un bit extra tal que, con $\oplus = \text{XOR}$:

- Paridad par: $P = b_0 \oplus b_1 \oplus \dots \oplus b_n$
- Paridad impar: $P = \text{NOT}(b_0 \oplus b_1 \oplus \dots \oplus b_n)$

Limitación clave: si se alteran un número **par** de bits, la paridad simple no detecta el error. La paridad horizontal + vertical (bidimensional, matriz de bits con paridad por fila y por columna) permite además **localizar** el bit errado (intersección fila/columna con paridad fallida).

3.4 ENTRELAZADO 2 DE 5

Código de distancia 2 donde cada palabra válida tiene exactamente dos 1's y tres 0's. Detecta errores de 1 bit (cambia la cantidad de 1's) pero **no puede corregir** porque no sabe cuál bit fue.

3.5 CÓDIGO DE HAMMING

Genera códigos de **distancia 3** → detecta y corrige 1 bit de error ($e=1 < d/2 = 1.5$).

Construcción para 4 bits de datos ($a_1 a_2 a_3 a_4$) + 3 de redundancia (p_1, p_2, p_3), con los p_i en las posiciones potencia de 2 (1, 2, 4):

$$p_1 = a_4 \oplus a_2 \oplus a_1 \quad p_2 = a_4 \oplus a_3 \oplus a_1 \quad p_3 = a_4 \oplus a_3 \oplus a_2$$

Al recuperar, se calculan los síndromes:

$$s_0 = p_1 \oplus a_4 \oplus a_2 \oplus a_1 \quad s_1 = p_2 \oplus a_4 \oplus a_3 \oplus a_1 \quad s_2 = p_3 \oplus a_4 \oplus a_3 \oplus a_2$$

$S = s_2s_1s_0$ en binario indica **qué posición** está errada (0 = sin error); se invierte ese bit para corregir.

Errores de tolerancia cero típicos acá: confundir la posición del bit (1-indexado, de derecha a izquierda) y equivocar cuáles a_i entran en cada p_i/s_i — conviene siempre reconstruir la tabla de columnas antes de calcular.

3.6 CHECKSUM

Se suman en binario todas las palabras del mensaje **despreciando el acarreo (carry) que se pierde por overflow**; el resultado se agrega al final. Al verificar, se repite la suma incluyendo el checksum recibido y se compara contra el valor esperado (todo 1's o el patrón que defina el protocolo).

4. Dónde practicar — Práctico 0 (Sistemas de Numeración)

Todos los ejercicios están resueltos en [Prácticos \(Soluciones\)/Práctico 0 \(Terminado\)/](#).

Ejercicio	Tema	Dificultad
Teóricas a-c	Justificación de métodos de conversión; criterio de finalización; relación bases 2/8	—
Ej 1	Conversiones directas base 10 $\leftrightarrow$ 2 $\leftrightarrow$ 16 (11 incisos)	★
Ej 2	Conversión a base 10 desde binario/hex	★
Ej 3	Conversión a base 10 con parte fraccionaria (binario, hex, base 13)	★
Ej 4 (OpenFing)	Conversión de base 10 con decimales a base 2 y 16	★★
Ej 5	Conversiones cruzadas (binario $\leftrightarrow$ hex, base 9 $\rightarrow$ 3, binario $\rightarrow$ octal)	★
Ej 6	Determinar en qué base(s) es válida cada operación aritmética dada	★★★
Ej 7	Sistema de ecuaciones resuelto trabajando en base 3	★★★
Ej 8 (complementario)	Conversión a/desde base 64	★★

Recomendación: hacé 1-5 primero (mecánica base), después 6-7 (los más conceptuales, típicos de “trampa” en examen), y 8 si te sobra tiempo.

5. Dónde practicar — Códigos y errores (Práctico 1, Ej. 1-2)

Aunque el Práctico 1 está agendado para la semana 2 en el cronograma, sus dos primeros ejercicios son puramente de esta semana:

Ejercicio	Tema	Dificultad
Ej 1	Hamming / paridad / entrelazado, capacidad de corrección	★★
Ej 2	Decodificar BCD vía Hamming de 7 bits	★

6. Dónde aparece en Parciales anteriores

No se identificó ningún ejercicio de parcial dedicado **solo** a conversión de bases pura o códigos de error — en los parciales revisados (2021-2025) estos temas no aparecen como ejercicio independiente, probablemente porque se los da por “herramienta básica” ya evaluada en el exa-

men. Sí aparece paridad bidimensional en:

- [Parciales/Taller_Preparacion_Primer_Parcial.pdf](#), Problema 2: paridad horizontal/vertical, función de detección/corrección de error en matriz 8×8 . **(sin resolver — para practicar solo)**

7. Dónde aparece en Exámenes anteriores

Examen	Ejercicio	Tema
solExAC202207.pdf	P4	Conversión de bases (base 64 $\leftrightarrow$ binario/hex)
solExAC202212.pdf	Problema 1	ROM que decodifica BCD $\rightarrow$ binario (usa noción de código)
solExAC202402.pdf	P1	Codificación y decodificación con Hamming (4 bits de datos)
solExAC202407.pdf	P3	Conversión de bases con aritmética de la base destino
solExAC202507-2.pdf	P2	Hamming (7,4) — codificación, corrección de 1 bit, ejemplo numérico completo
solExAC202512.pdf	Problema 1	ROM que calcula distancia de Hamming entre dos tiras de 16 bits (combina M1+M3)

Patrón: la conversión de bases pura aparece como pregunta corta (P3/P4) en ~ 2 de cada 6-7 exámenes; los códigos de Hamming aparecen con más frecuencia y a veces se combinan con diseño de ROM (tema de la semana 4) — vale la pena tenerlos sólidos desde ya porque reaparecen indirectamente más adelante.

8. Errores frecuentes a vigilar (tolerancia cero)

1. **Conversión con fraccionarios:** olvidar que el Caso B para fraccionarios multiplica (no divide), y confundir cuántos dígitos pide el enunciado.
2. **Bases desconocidas:** olvidar verificar que los símbolos usados sean válidos para la base propuesta.
3. **Hamming:** invertir el orden de bits (posición 1 = bit menos significativo, contando desde la derecha) o mezclar qué a_i entra en cada p_i .

4. **Paridad:** confundir paridad par con impar, o no notar que un número par de bits alterados es invisible a la paridad simple.
5. **Checksum:** olvidar despreciar el acarreo/overflow al sumar.